

2. Overview of Statistical Learning

What is statistical learning ?

Why estimate f ?

How to estimate f ?

Prediction accuracy and model interpretability

Supervised v.s. Unsupervised

Regression v.s. Classification

Assessing model accuracy

Quality of fit

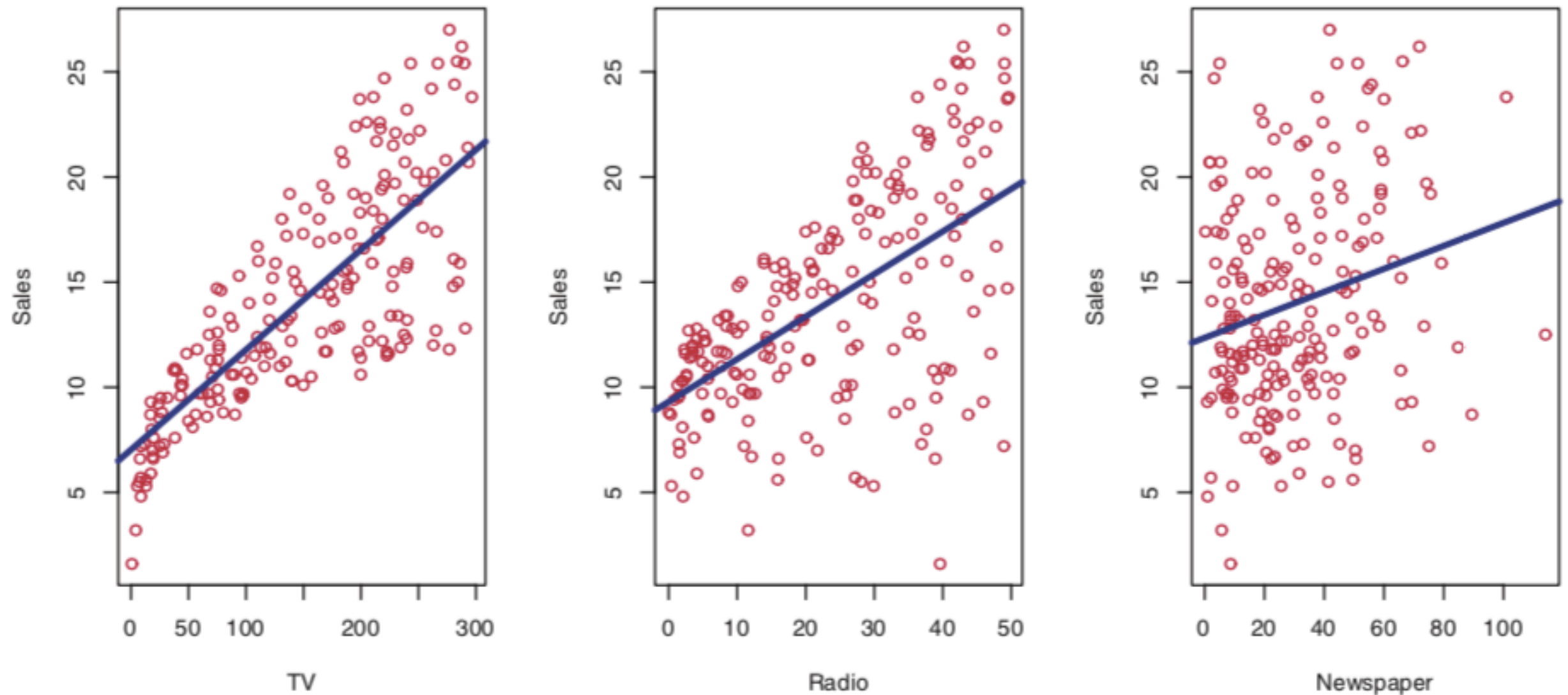
Bias and variance trade-off

Classification setting

2.1 What is Statistical Learning

Why estimate f ?

Example *Advertising dataset*



Y
output: sales

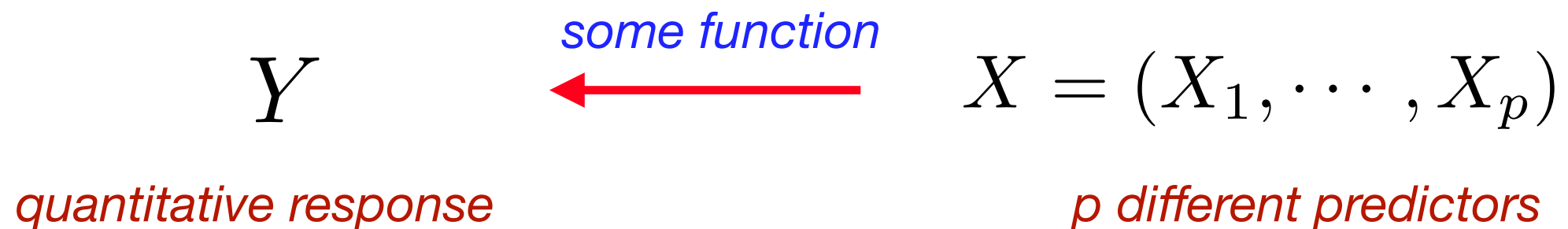


(X_1, X_2, X_3)
input: advertising on TV, Radio, Newspaper

2.1 What is Statistical Learning

Why estimate f ?

General regression setting



$$Y = f(X) + \epsilon.$$

f is some fixed but unknown function of $X_1, \dots, X_p$.

ϵ is a random error term, which is independent of X and has mean 0

2.1 What is Statistical Learning

Why estimate f ?

Example *Income and Education*

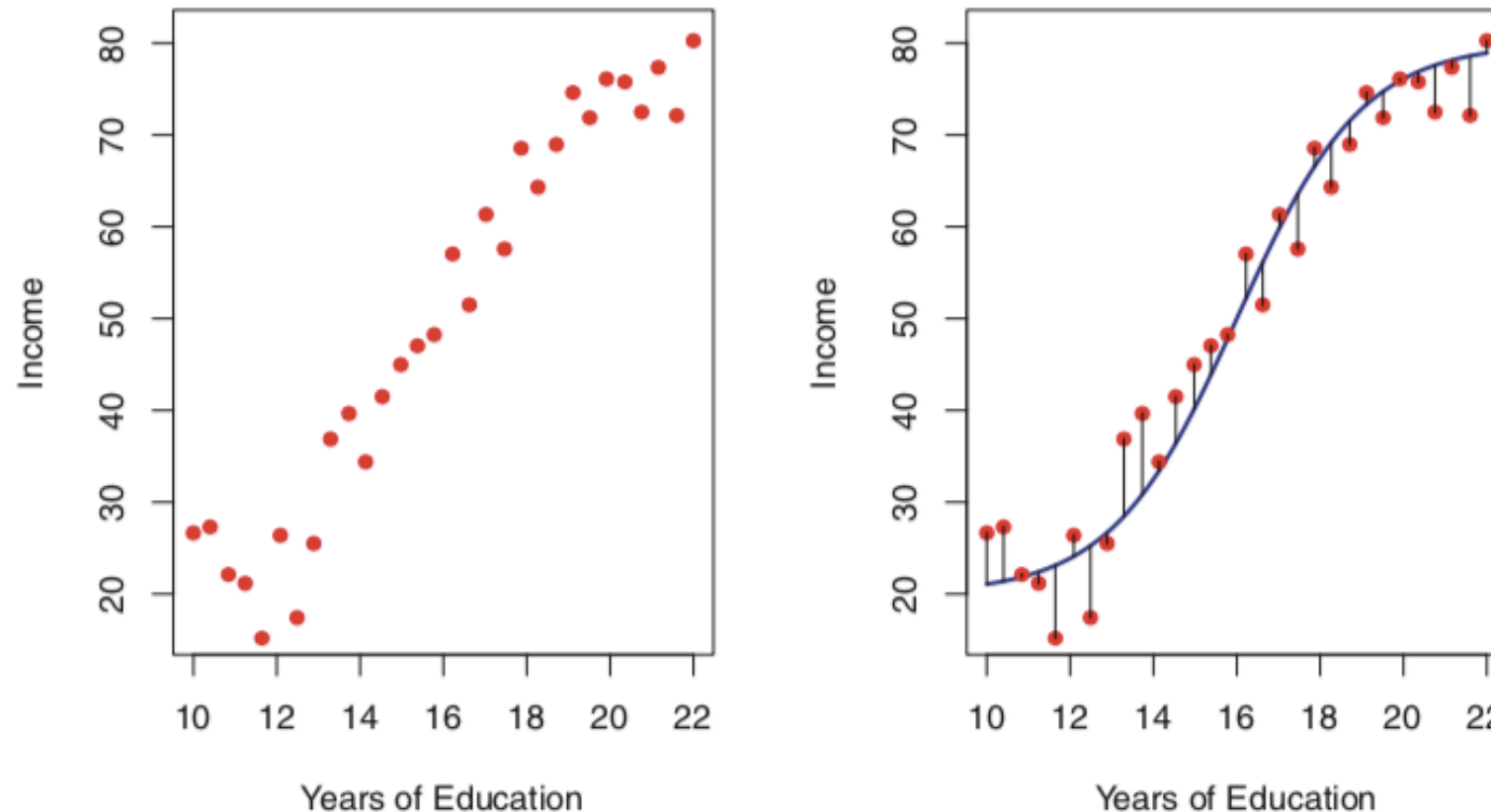


FIGURE 2.2. The **Income** data set. Left: The red dots are the observed values of **income** (in tens of thousands of dollars) and **years of education** for 30 individuals. Right: The blue curve represents the true underlying relationship between **income** and **years of education**, which is generally unknown (but is known in this case because the data were simulated). The black lines represent the error associated with each observation. Note that some errors are positive (if an observation lies above the blue curve) and some are negative (if an observation lies below the curve). Overall, these errors have approximately mean zero.

2.1 What is Statistical Learning

Why estimate f ?

Task 1: Prediction

Given an input X without output, estimating f can predict the output.

$$\hat{Y} = \hat{f}(X)$$

prediction of Y *estimate of $f(\cdot)$ based on data $(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)$*

Example

If a store spends 10K on TV advertising, what will its sales be?

Example

If a father's height is 170cm, what will his adult son's height be?

2.1 What is Statistical Learning

Why estimate f ?

Task 1: Prediction

Discussion of Randomness

Recall that $\hat{f}(\cdot)$ is an estimate of function $f(\cdot)$ based on data $(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)$.

- If we view the data as given, i.e, already determined numbers, then $\hat{f}(\cdot)$ is also determined. There is no randomness for $\hat{f}(\cdot)$. From this perspective, we are viewing $\hat{f}(\cdot)$ as an **instance**.
- On the other hand, if we view $\hat{f}(\cdot)$ as a **methodology**, e.g., every day thousands of different people apply this method to different datasets, then our purpose is to investigate the property of this **methodology in the future**. From this perspective, the future data $(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)$ is viewed as random (*because there is random noise*), as a result, the estimated $\hat{f}(\cdot)$ based on this future data is also viewed as random. This is the so-called **generalization capability** of a machine learning method.

When we are interested in the property of $\hat{f}(\cdot)$, oftentimes, we view it as being random.

2.1 What is Statistical Learning

Why estimate f ?

Task 1: Prediction

accuracy of $\hat{Y}$ as a prediction for Y

What is the randomness here?

$$\begin{aligned}
 E(Y - \hat{Y})^2 &= E[f(X) + \epsilon - \hat{f}(X)]^2 \\
 &= E \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible}},
 \end{aligned}$$

expected squared error

$$E(Y - \hat{Y}) = E[f(X) + \epsilon - \hat{Y}] = \underbrace{f(X) - E\hat{f}(X)}_{\text{Bias}}$$

*For prediction, the actual form of $f(X)$, $\hat{f}(X)$ does not matter.
Only the accuracy of prediction matters.*

2.1 What is Statistical Learning

Why estimate f ?

Task 2: Inference *actual form of $f(X)$, $\hat{f}(X)$ very matters.*

Under the relationship between Y and $X = (X_1, \dots, X_p)$

$\hat{f}$ can not be treated as a black box. Its exact form matters.

- *Which predictors are associated with the response?* It is often the case that only a small fraction of the available predictors are substantially associated with Y . Identifying the few *important* predictors among a large set of possible variables can be extremely useful, depending on the application.
- *What is the relationship between the response and each predictor?* Some predictors may have a positive relationship with Y , in the sense that increasing the predictor is associated with increasing values of Y . Other predictors may have the opposite relationship.

2.1 What is Statistical Learning

Why estimate f ?

Task 2: Inference

Example (Advertising on TV, Radio, Newspaper)

- *Which media contribute to sales?*
- *Which media generate the biggest boost in sales? or*
- *How much increase in sales is associated with a given increase in TV advertising?*

actual form of $f(X)$, $\hat{f}(X)$ very matters.

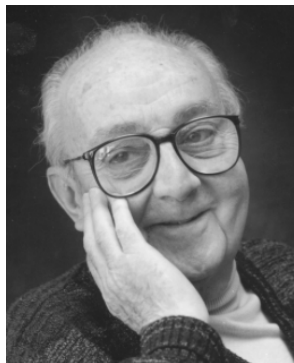
2.1 What is Statistical Learning

How to estimate f ?

Parametric method *Linear regression (linear model)*

Assume f is linear in X

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p.$$



“All models are wrong, but some are useful.”

— — George E.P. Box

Prediction

$$\hat{Y} = \hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \hat{\beta}_2 X_2 + \dots + \hat{\beta}_p X_p$$

Inference

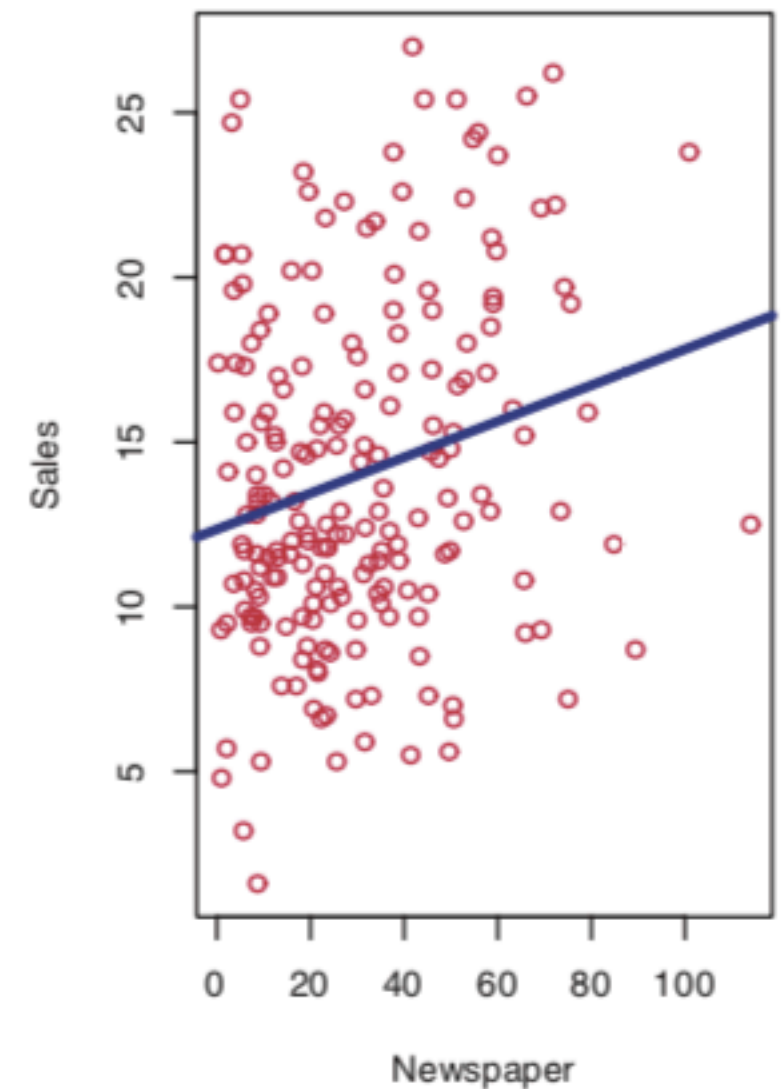
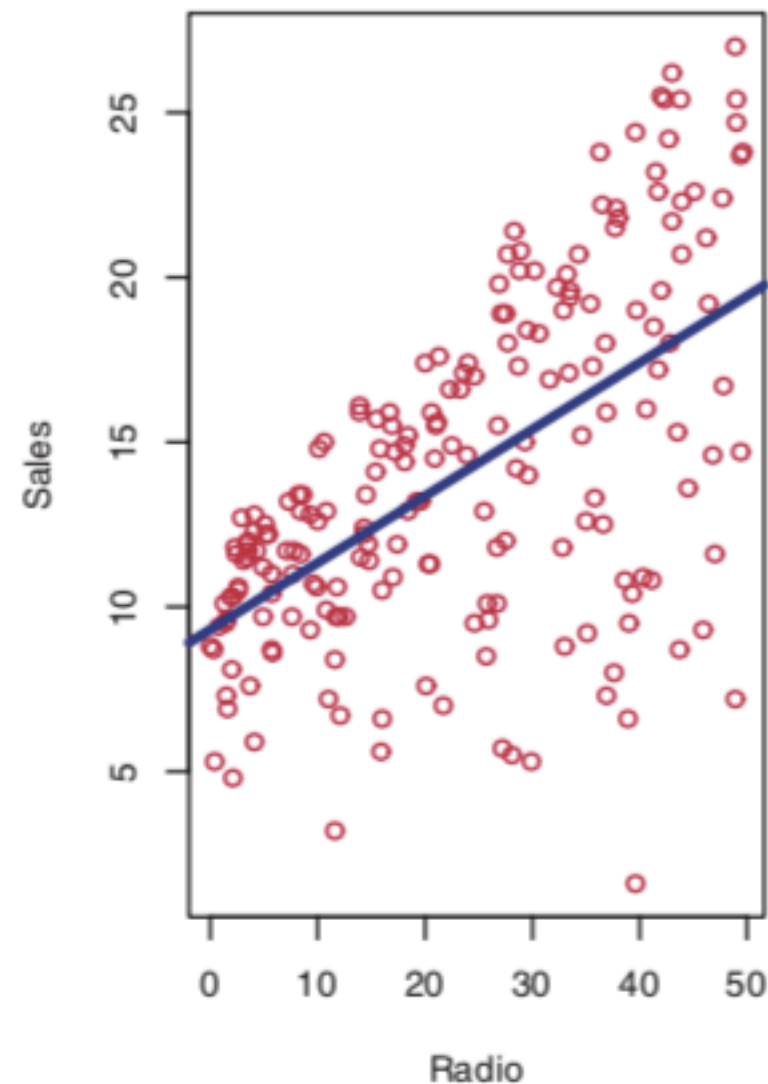
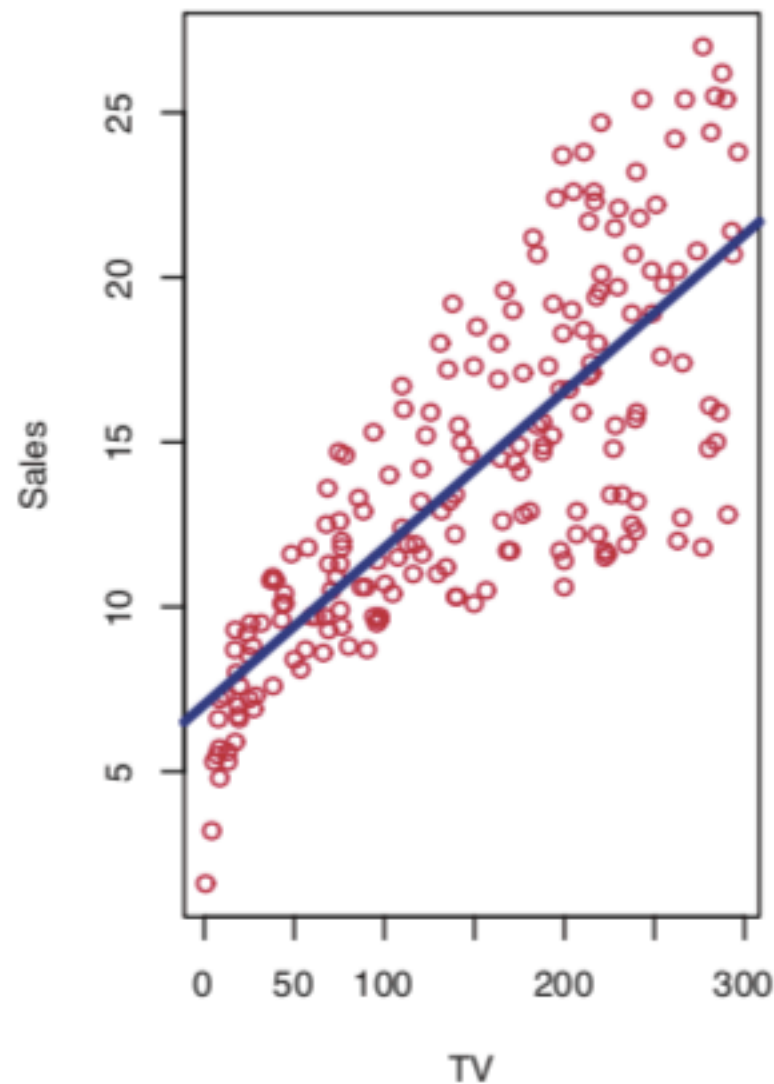
$$\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2, \dots, \hat{\beta}_p$$

2.1 What is Statistical Learning

How to estimate f ?

Parametric method *Linear regression (linear model)*

Example Advertising dataset

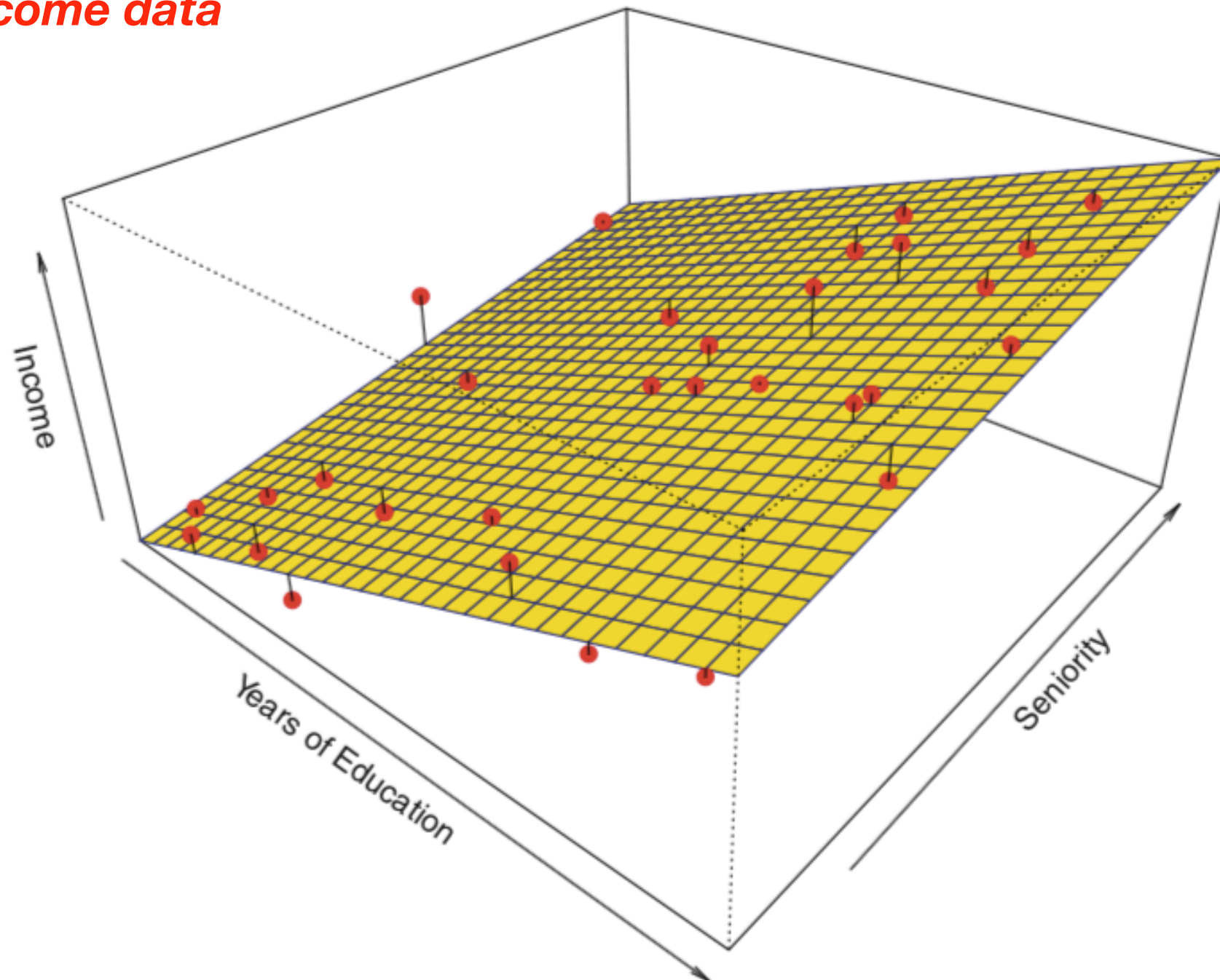


2.1 What is Statistical Learning

How to estimate f ?

Parametric method *Linear regression (linear model)*

Example Income data



2.1 What is Statistical Learning

How to estimate f ?

Parametric method *Non-linear regression (non-linear model)*

Assume f is a generalized linear model (GLM)

$$f(X) = h(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)$$

a link function 

Popular choices: $h(u) = \exp(u)$; $h(u) = 1/(1 + e^{-u})$

Prediction

$$\hat{Y} = \hat{f}(X) = h(\hat{\beta}_0 + \hat{\beta}_1 X_1 + \cdots + \hat{\beta}_p X_p)$$

Inference

$$\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2, \cdots, \hat{\beta}_p$$

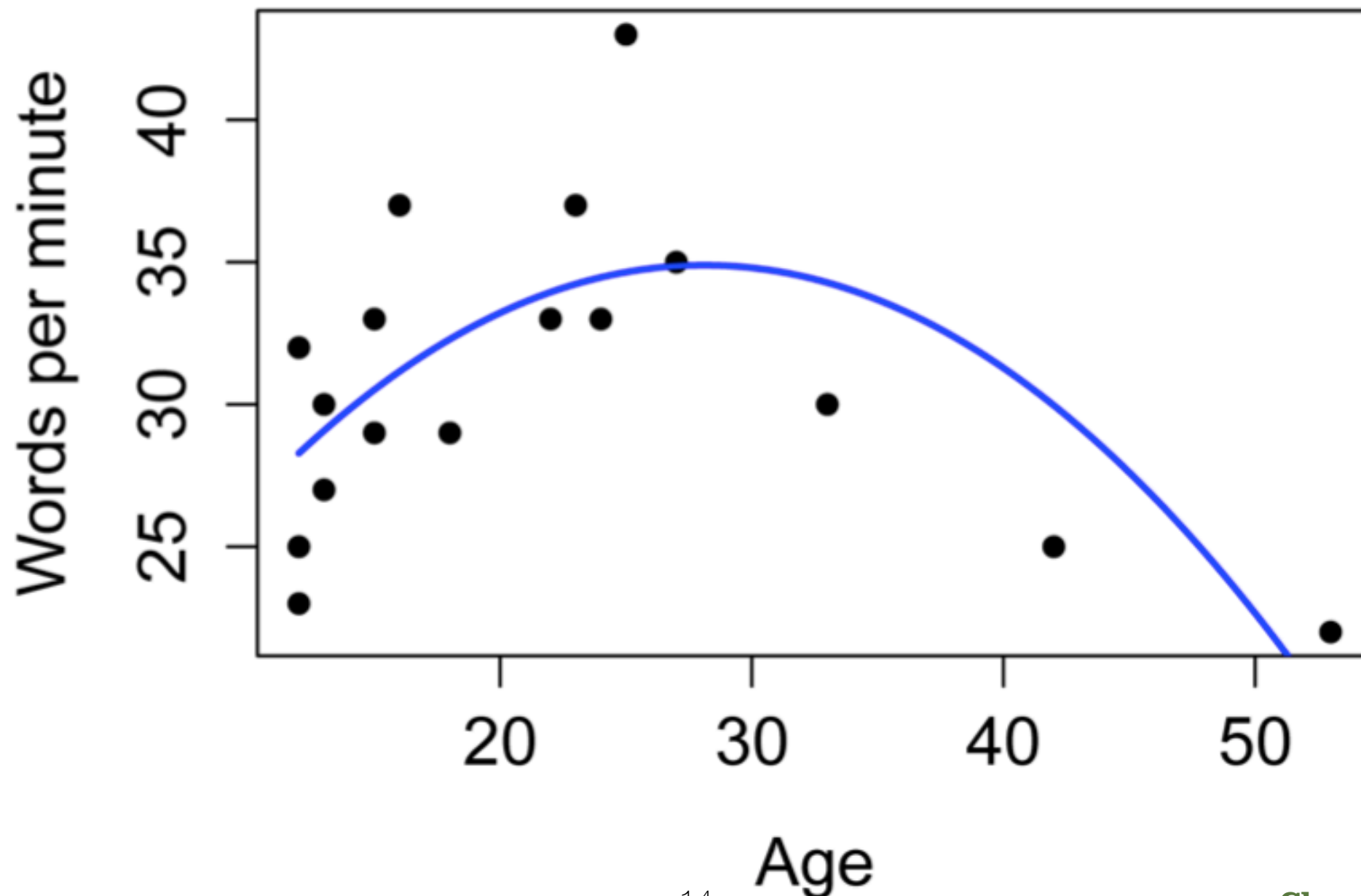
Computationally more difficult to solve than linear model.

2.1 What is Statistical Learning

How to estimate f ?

Parametric method *Non-linear regression (non-linear model)*

Example Typing speed and age



2.1 What is Statistical Learning

How to estimate f ?

Non-Parametric method *try to fit the data*

No explicit assumptions on functional form of f

K-nearest neighbor (KNN)

$$\hat{f}(X) = K^{-1} \sum_{i \in \text{KNN}(X)} Y_i$$

here K is a positive integer

Kernel smoothing

$$\hat{f}(X) = \frac{1}{nh^p} \cdot \sum_{i=1}^n K\left(\frac{X - X_i}{h}\right) Y_i$$

here K is a kernel function

Pros

*needs less data
easy inference*

Con

*the particular function
form can be totally wrong*

Parametric method

Non-Parametric method

*fit a wider range
of possible shapes of f*

*needs a lot of data
hard to inference*

2.1 What is Statistical Learning

```
##### KNN regression, fitted curve with different K value, report training error and test error

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.neighbors import KNeighborsRegressor

# Generate synthetic data
np.random.seed(0)
x = np.sort(np.random.rand(100) * 10)
y = np.sin(x) + np.random.normal(scale=0.5, size=x.shape)

# Reshape x for sklearn
x = x.reshape(-1, 1)

# Split the data into training and test sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

# Plot the original training data
plt.scatter(x_train, y_train, label='Training Data', color='lightgray')

# K values to try
k_values = [1, 5, 10, 20]
colors = ['red', 'green', 'blue', 'black']

# Fit KNN regression for different K values
for k, color in zip(k_values, colors):
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(x_train, y_train)

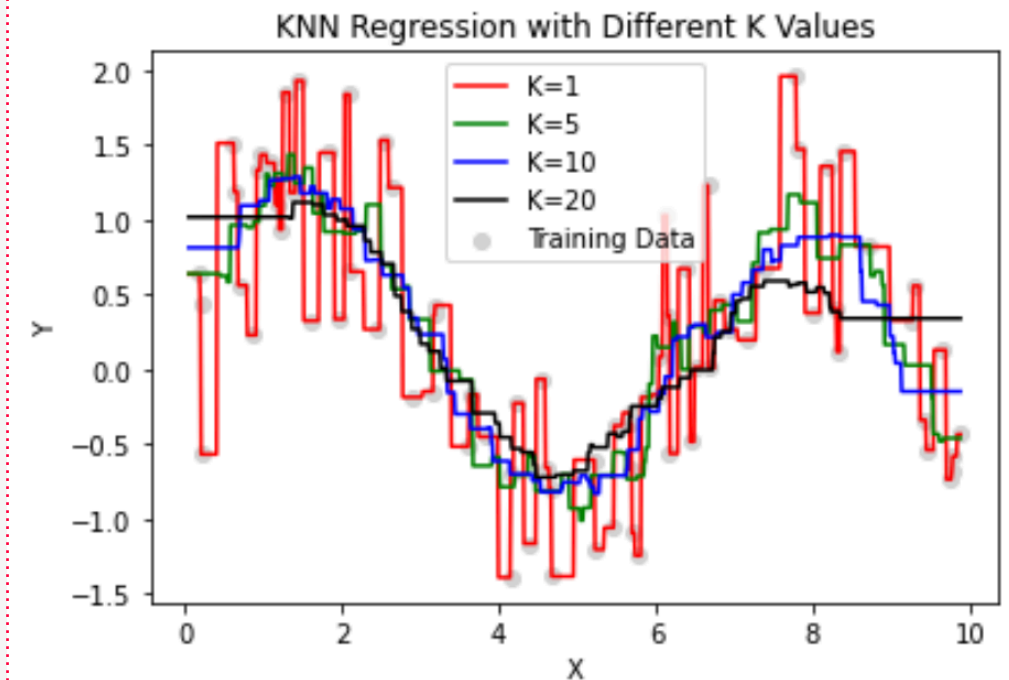
    # Predict on a smooth range for plotting
    xs = np.linspace(x.min(), x.max(), 1000).reshape(-1, 1)
    ys = knn.predict(xs)
    plt.plot(xs, ys, label=f'K={k}', color=color)

    # Calculate and report training error
    y_train_pred = knn.predict(x_train)
    training_error = mean_squared_error(y_train, y_train_pred)
    print(f'K={k}: Training Error = {training_error:.4f}')

    # Calculate and report test error
    y_test_pred = knn.predict(x_test)
    test_error = mean_squared_error(y_test, y_test_pred)
    print(f'K={k}: Test Error = {test_error:.4f}')

# Customize plot
plt.xlabel('X')
plt.ylabel('Y')
plt.title('KNN Regression with Different K Values')
plt.legend()
plt.show()
```

K=1: Training Error = 0.0000
 K=1: Test Error = 0.5778
 K=5: Training Error = 0.2495
 K=5: Test Error = 0.3391
 K=10: Training Error = 0.2672
 K=10: Test Error = 0.3121
 K=20: Training Error = 0.3665
 K=20: Test Error = 0.3426



2.1 What is Statistical Learning

How to estimate f ?

Non-Parametric method *spline method*

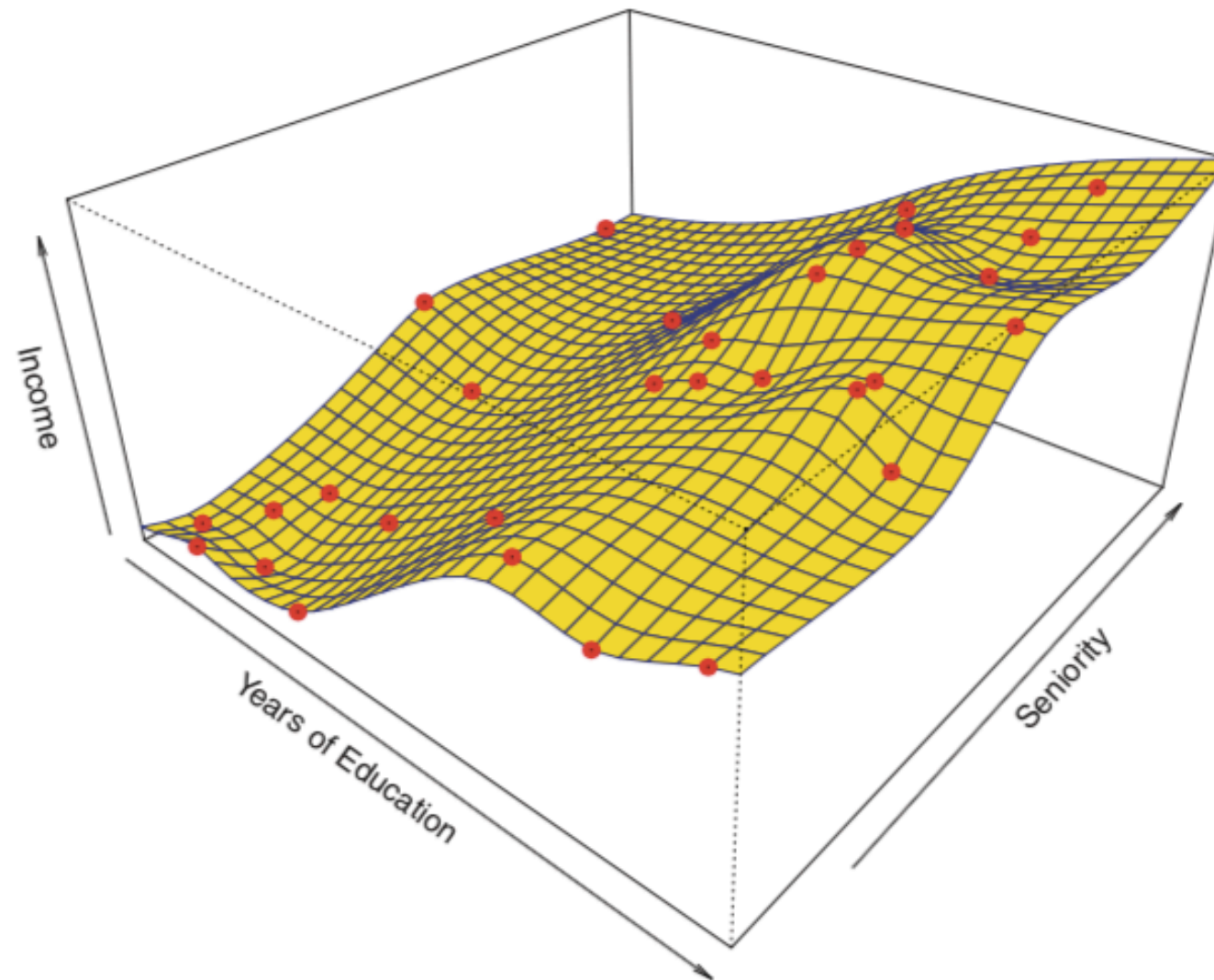


FIGURE 2.6. A rough thin-plate spline fit to the **Income** data from Figure 2.3. This fit makes zero errors on the training data.

2.1 What is Statistical Learning

Trade-off between prediction accuracy and model interpretability

Linear regression

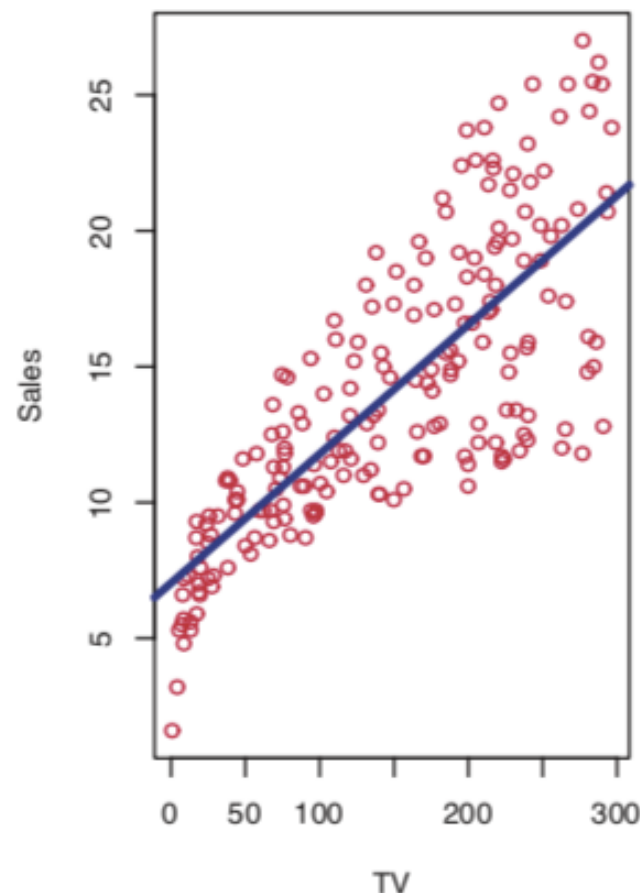
*prediction accuracy is low
model is inflexible*

restrictive models are more interpretable

Spline method

*prediction accuracy is high
model is flexible*

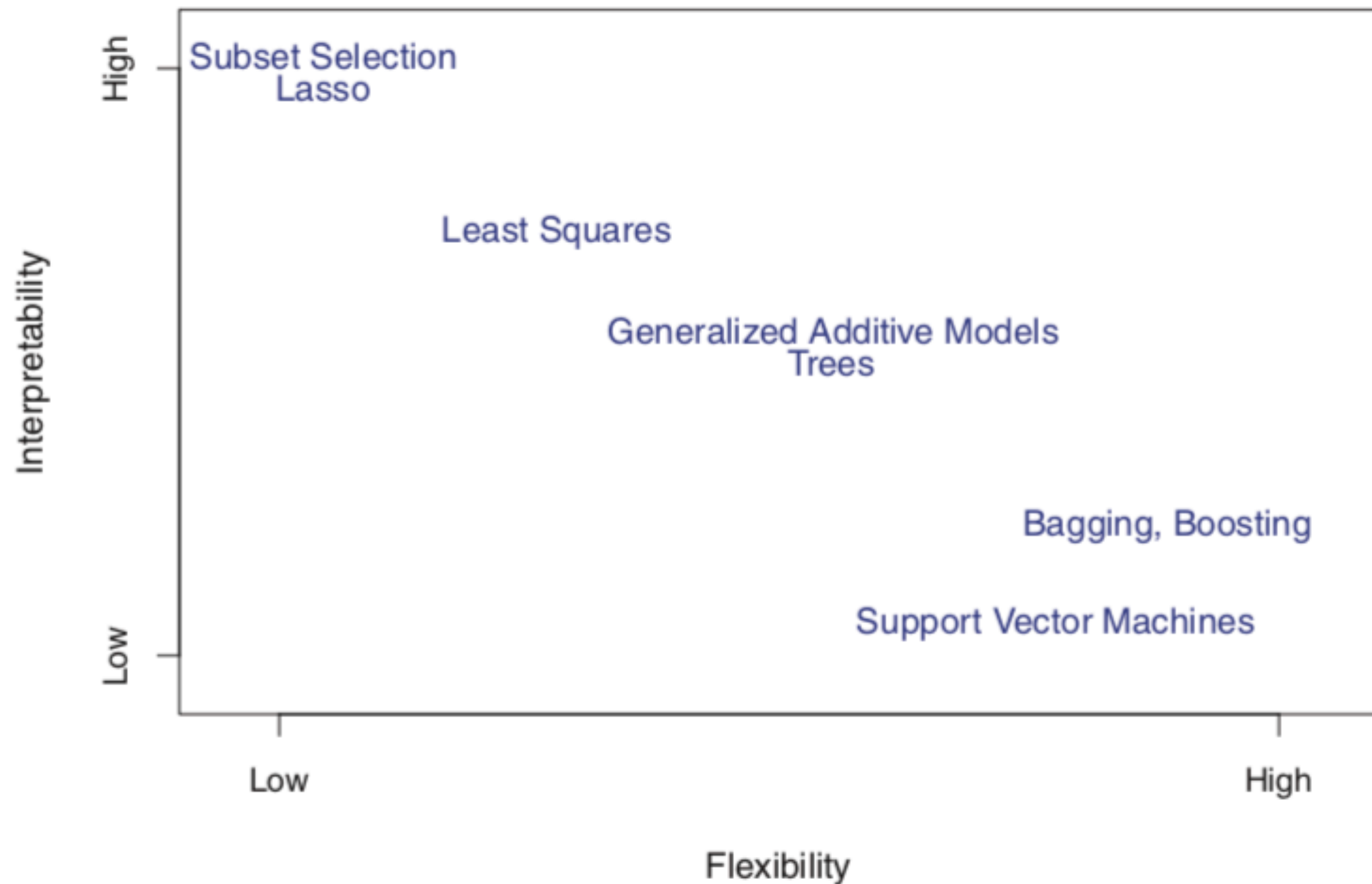
less interpretable



***prediction accuracy is low
but easy to interpret***

2.1 What is Statistical Learning

Trade-off between prediction accuracy and model interpretability



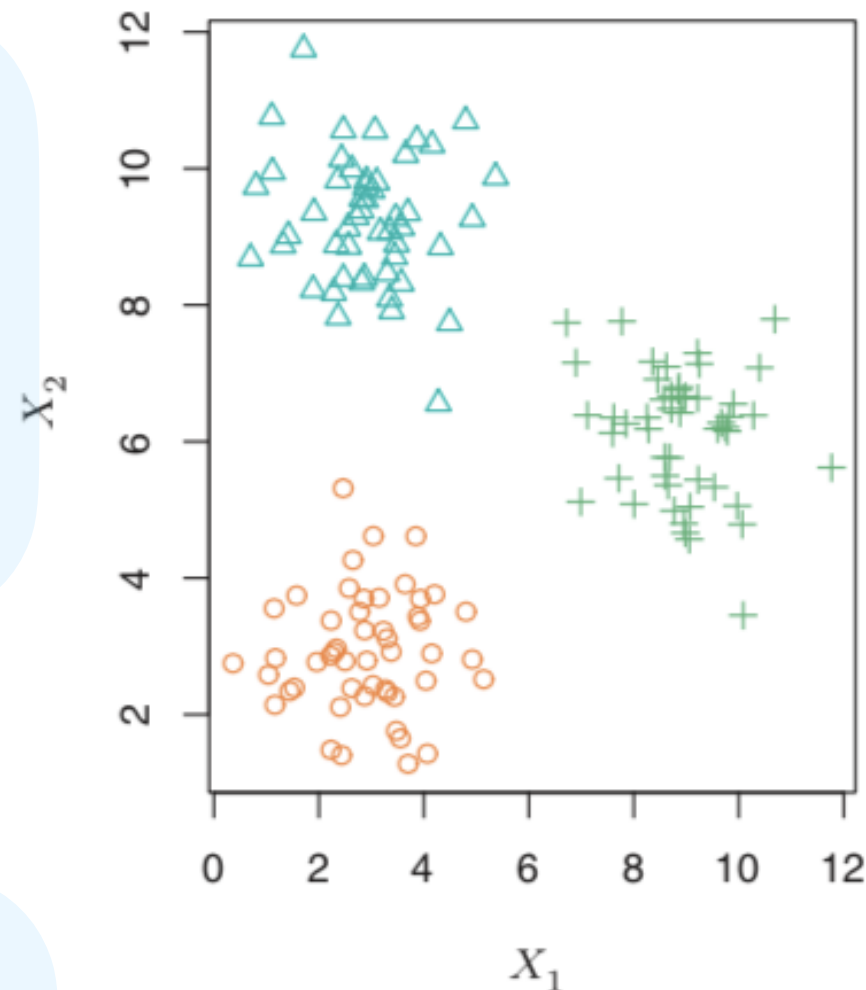
2.1 What is Statistical Learning

Supervised v.s. Unsupervised

Supervised Learning

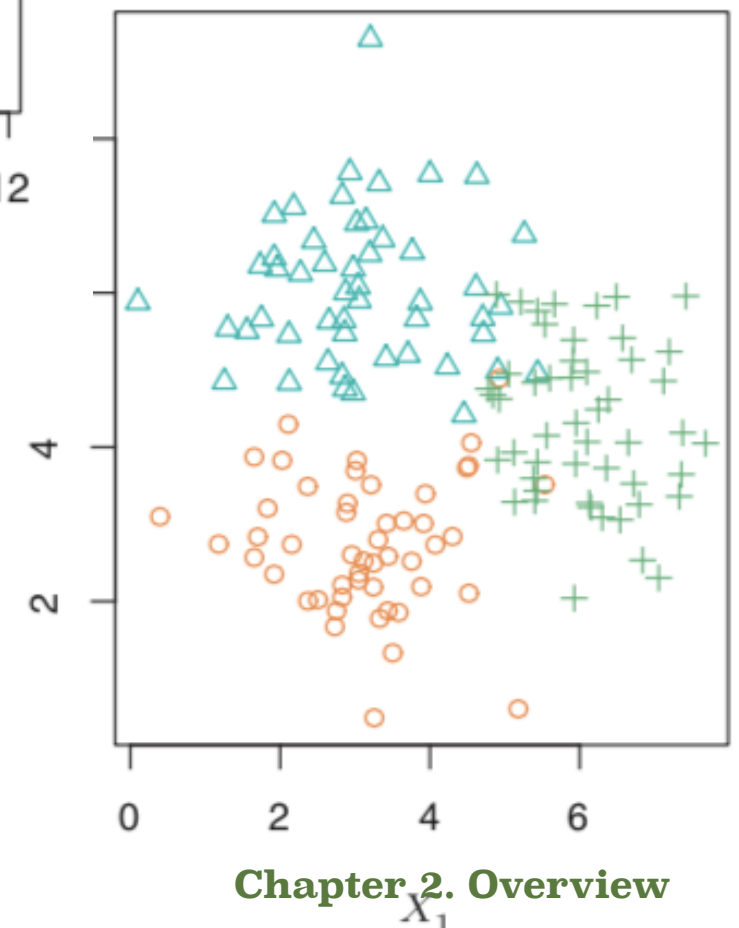
Each X has an associated Y

linear regression
logistic regression
GAM
boosting
SVM



Unsupervised Learning

Only X but no Y
clustering



2.1 What is Statistical Learning

Supervised v.s. Unsupervised

K-means clustering, with clustering results shown clustering results

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from scipy.optimize import linear_sum_assignment
from sklearn.metrics import accuracy_score

# Generate synthetic data
np.random.seed(0)
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

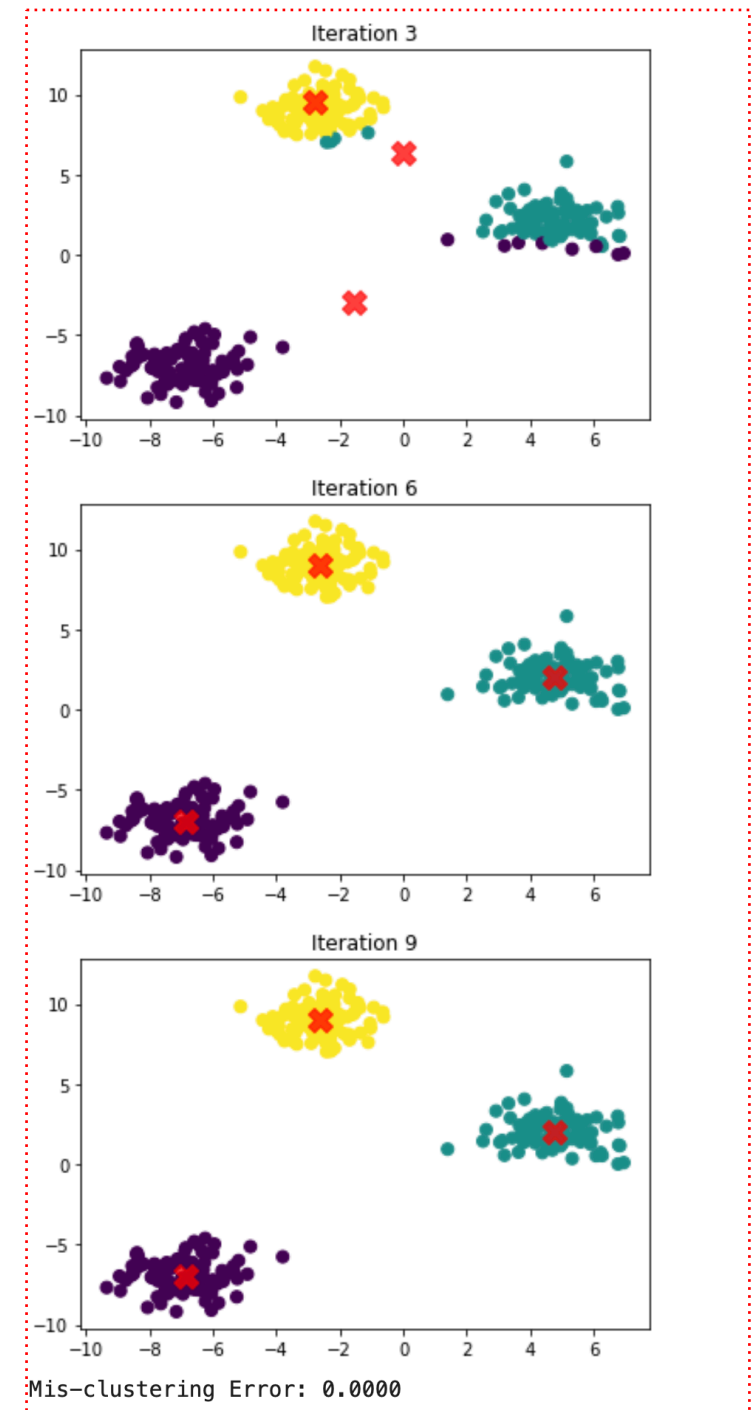
# Define a function to plot clusters
def plot_clusters(X, y_pred, centroids, iteration):
    plt.scatter(X[:, 0], X[:, 1], c=y_pred, s=50, cmap='viridis')
    plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, alpha=0.75, marker='X')
    plt.title(f'Iteration {iteration}')
    plt.show()

# Function to calculate mis-clustering error
def calculate_misclustering_error(y_true, y_pred):
    cost_matrix = np.zeros((3, 3))
    for i in range(3):
        for j in range(3):
            cost_matrix[i, j] = np.sum((y_true == i) & (y_pred == j))
    row_ind, col_ind = linear_sum_assignment(cost_matrix, maximize=True)
    return 1 - accuracy_score(y_true, [col_ind[i] for i in y_pred])

# Initialize KMeans
kmeans = KMeans(n_clusters=3, init='random', n_init=1, max_iter=5, random_state=42, algorithm='full')

# Run K-means and plot every five iterations
iteration = 0
while iteration < 9:
    kmeans.fit(X)
    y_pred = kmeans.labels_
    centroids = kmeans.cluster_centers_
    plot_clusters(X, y_pred, centroids, iteration + 3)
    iteration += 3
    kmeans.max_iter += 3

# Calculate mis-clustering error
mis_clustering_error = calculate_misclustering_error(y_true, y_pred)
print(f'Mis-clustering Error: {mis_clustering_error:.4f}')
```



2.1 What is Statistical Learning

Classification v.s. Clustering ?

2.1 What is Statistical Learning

Regression and Classification

Regression

response Y is quantitative

Linear regression

Generalized linear model

K nearest neighbor

Boosting

Non-parametric regression

Kernel method

Classification

response Y is categorical

K nearest neighbor

Boosting

Support Vector Machine

Naive Bayes

Logistic regression

2.2 Assessing Model Accuracy

No Free Lunch

No single method dominates all others on all datasets

Strength of a method depends on whether its assumptions are true



hard to check

How to compare the performance of different methods ?

2.2 Assessing Model Accuracy

Measuring quality of fit

Training error is one measurement of the quality of fit

Training MSE (mean squared error)

$$\text{Training MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 \quad \text{for the training data } (x_i, y_i), i = 1, \dots, n.$$

Rationale of Training MSE

*If we view $\hat{f}$ as being independent of $\{(x_i, y_i)\}_{i=1}^n$ (of course, this is untrue), by the **law of large numbers**, we know as $n \rightarrow \infty$, i.e., the data is very plentiful, then*

$$\text{Training MSE} \longrightarrow E[(\hat{f}(X) - Y)^2]$$

Here we also view $X_1, \dots, X_n$ as being randomly sampled from a certain distribution.

2.2 Assessing Model Accuracy

Measuring quality of fit

The rationale of training MSE holds only when $\hat{f}(\cdot)$ is independent of $\{(x_i, y_i)\}_{i=1}^n$. This is certainly untrue because $\hat{f}(\cdot)$ is **trained** on this data. So what issues can this dependence cause?

Recall that, in our fundamental framework, we are assuming the data $y_i = f(x_i) + \varepsilon_i$ comprising two parts: **information part** $f(x_i)$ and **noise part** ε_i . Essentially, in the process of machine learning, the estimated $\hat{f}(\cdot)$ *not only try to fit the information part but also tries to fit the noise part!* Clearly, the better $\hat{f}(\cdot)$ fits the data, **the more it fits the noise**, called **overfitting**. This tells us that if your estimated $\hat{f}(\cdot)$ fits too well on the data, you should be careful.

This also implies that using training MSE to measure the quality of estimated $\hat{f}(\cdot)$ is **unfair** because more complicated function can fit better on the training data, but it very likely fits too much on the noise making it actually not accurate for prediction.

Setting $\hat{f}(x_i) = y_i$ leading to the training MSE = 0.

called interpolation

↑
Overfitting

2.2 Assessing Model Accuracy

Measuring quality of fit

It shows training MSE is not the best measurement of the model accuracy.

The ultimate measurement is the test MSE.

$$\text{Test MSE} = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}}^n (y_j - \hat{f}(x_j))^2$$

where $\mathcal{J}$ refers to the test set of data, and $\hat{f}$ is obtained from the training data.

Note $\hat{f}(\cdot)$ is independent of test data.

Methods, such as cross validation, AIC, BIC, are often employed to compute or estimate the size of the test error

2.2 Assessing Model Accuracy

Measuring quality of fit

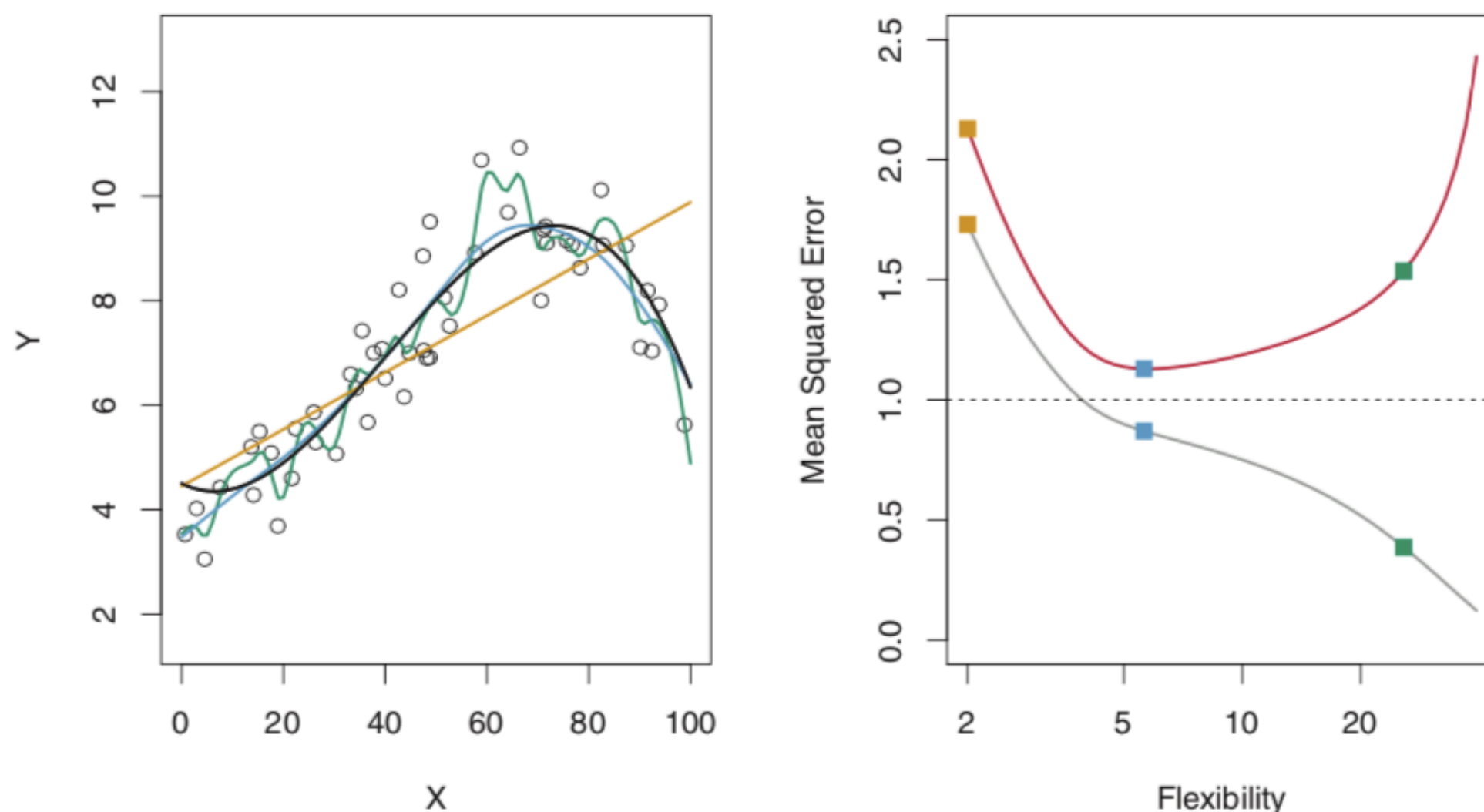


FIGURE 2.9. Left: Data simulated from f , shown in black. Three estimates of f are shown: the linear regression line (orange curve), and two smoothing spline fits (blue and green curves). Right: Training MSE (grey curve), test MSE (red curve), and minimum possible test MSE over all methods (dashed line). Squares represent the training and test MSEs for the three fits shown in the left-hand panel.

Just imagine that the more flexible $\hat{f}$ is, the more complicate (mathematically) it is.

2.2 Assessing Model Accuracy

Measuring quality of fit

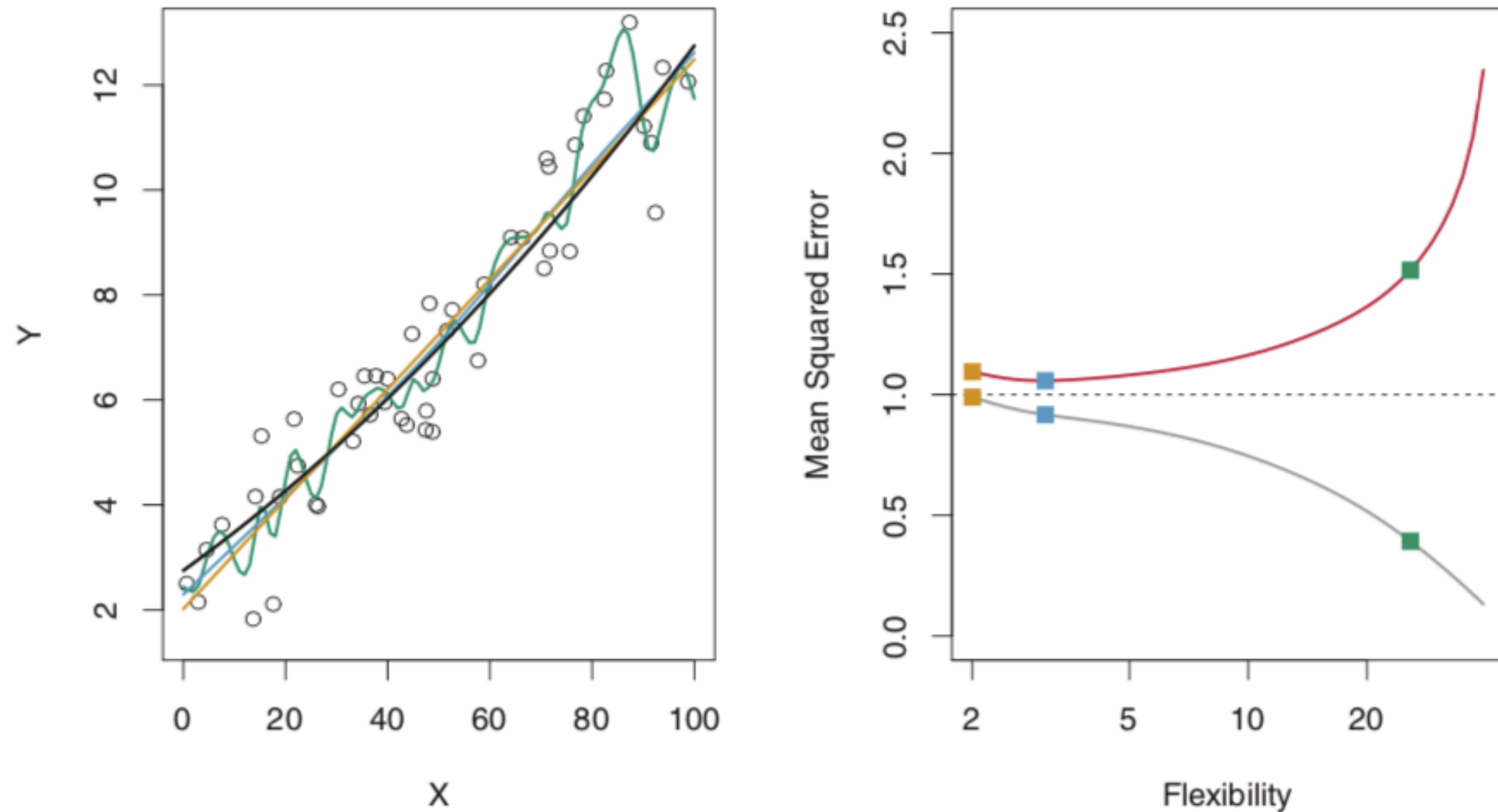


FIGURE 2.10. Details are as in Figure 2.9, using a different true f that is much closer to linear. In this setting, linear regression provides a very good fit to the data.

2.2 Assessing Model Accuracy

Measuring quality of fit

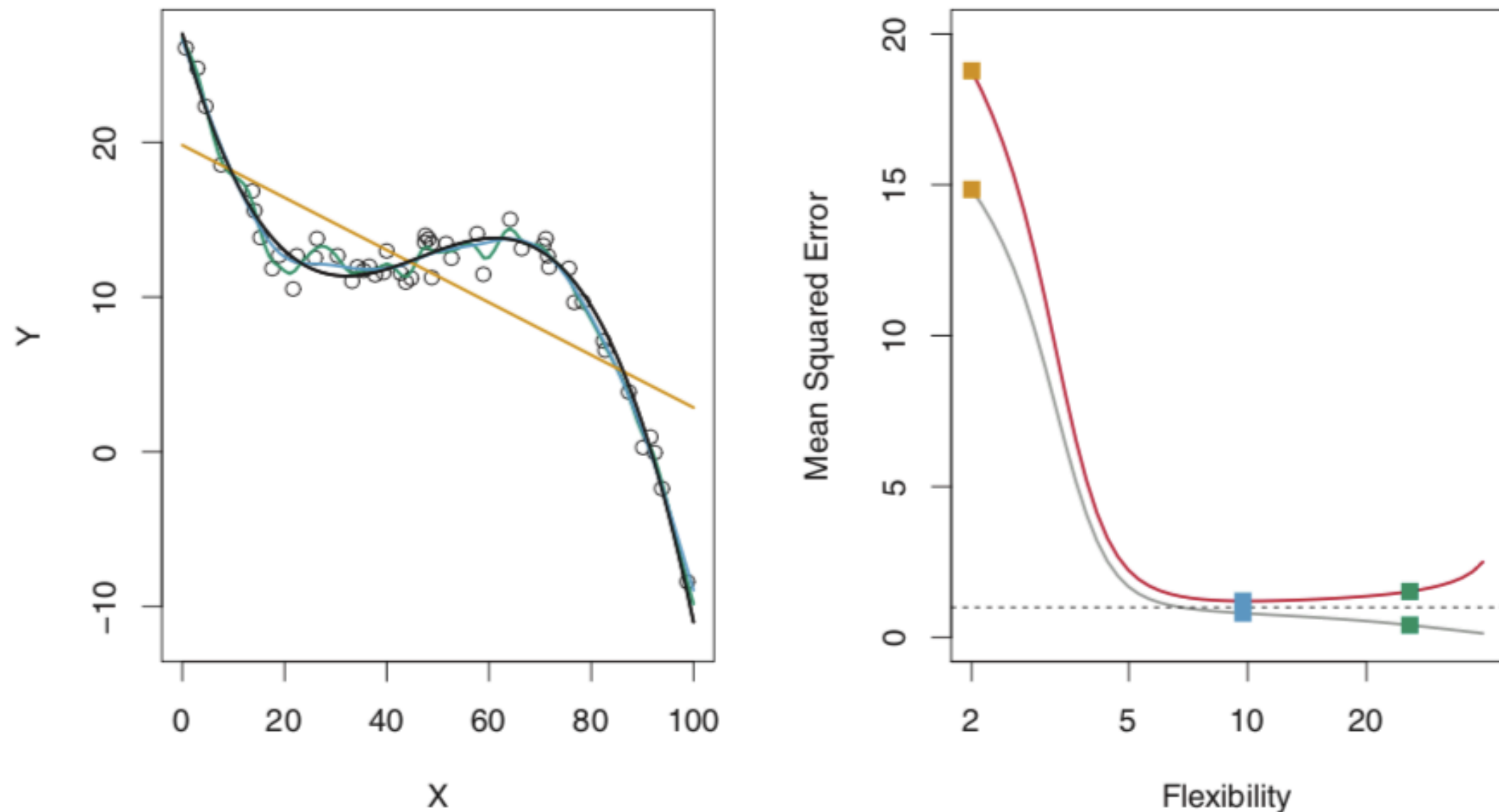


FIGURE 2.11. Details are as in Figure 2.9, using a different f that is far from linear. In this setting, linear regression provides a very poor fit to the data.

2.2 Assessing Model Accuracy

Bias and Variance Trade-off

Expected test MSE

for a given value x_0

What is the randomness here?

$$E \left(y_0 - \hat{f}(x_0) \right)^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon).$$

*expected test error
desired to minimize*

Variance

Bias

expected test MSE can never lie below $\text{Var}(\epsilon)$, the irreducible error

more noise is fitted

better captures underlying pattern

Method is more flexible: variance increases and bias decreases

Method is simpler (less flexible): introduce higher bias

2.2 Assessing Model Accuracy

Bias and Variance Trade-off

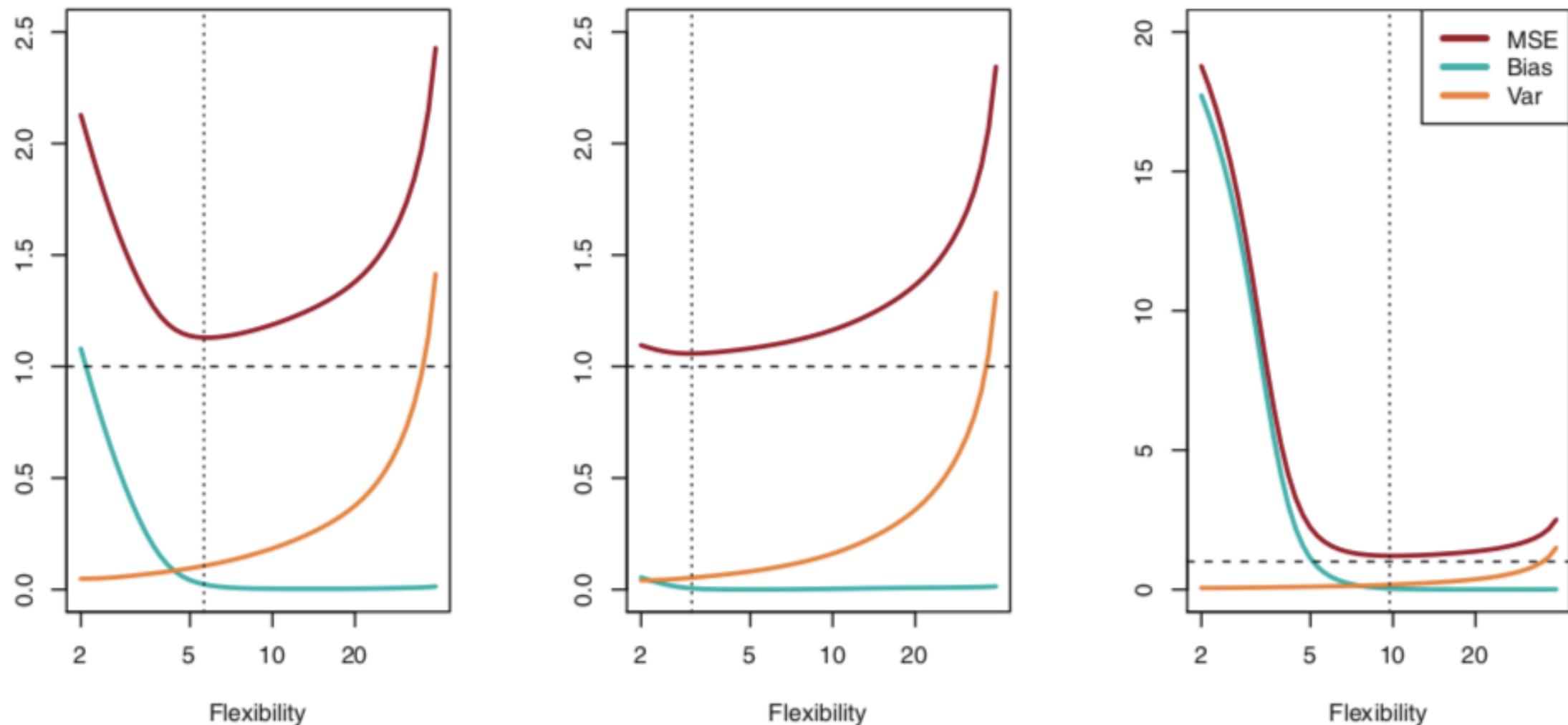


FIGURE 2.12. Squared bias (blue curve), variance (orange curve), $\text{Var}(\epsilon)$ (dashed line), and test MSE (red curve) for the three data sets in Figures 2.9–2.11. The vertical dotted line indicates the flexibility level corresponding to the smallest test MSE.

2.2 Assessing Model Accuracy

```
##### spline regression with different smooth levels and corresponding training error and test error
```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import UnivariateSpline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Generate synthetic data
np.random.seed(0)
x = np.sort(np.random.rand(100) * 10)
y = np.sin(x) + np.random.normal(scale=0.5, size=x.shape)

# Split the data into training and test sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

sorted_indices = np.argsort(x_train)
x_train = x_train[sorted_indices]
y_train = y_train[sorted_indices]

# Plot the original training data
plt.scatter(x_train, y_train, label='Training Data', color='lightgray')

# Fit smooth regression curves with different smoothness levels
smoothness_levels = [5, 20, 100]
colors = ['red', 'green', 'blue']

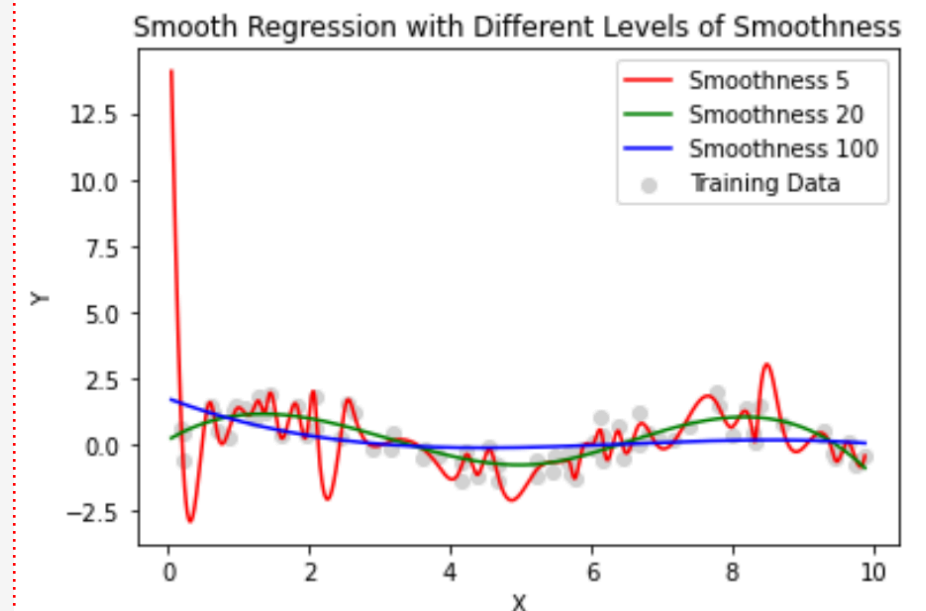
for s, color in zip(smoothness_levels, colors):
    spline = UnivariateSpline(x_train, y_train, s=s)
    xs = np.linspace(x.min(), x.max(), 1000)
    plt.plot(xs, spline(xs), label=f'Smoothness {s}', color=color)

    # Calculate and report training error
    y_train_fit = spline(x_train)
    training_error = mean_squared_error(y_train, y_train_fit)
    print(f'Smoothness {s}: Training Error = {training_error:.4f}')

    # Calculate and report test error
    y_test_fit = spline(x_test)
    test_error = mean_squared_error(y_test, y_test_fit)
    print(f'Smoothness {s}: Test Error = {test_error:.4f}')

# Customize plot
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Smooth Regression with Different Levels of Smoothness')
plt.legend()
plt.show()
```

Smoothness 5: Training Error = 0.0625
 Smoothness 5: Test Error = 11.5916
 Smoothness 20: Training Error = 0.2501
 Smoothness 20: Test Error = 0.2241
 Smoothness 100: Training Error = 0.5761
 Smoothness 100: Test Error = 0.7214



2.2 Assessing Model Accuracy

Classification error

y_i is categorical, i.e., $\{-1, +1\}$

Training error rate

training data $(x_1, y_1), \dots, (x_n, y_n)$, y_i s are labels

$$n^{-1} \sum_{i=1}^n I(\hat{y}_i \neq y_i) \quad \text{where } \hat{y}_i = \hat{f}(x_i)$$

 *misclassified*

Test error rate

test data $\mathcal{J}$

$$|\mathcal{J}|^{-1} \sum_{j \in |\mathcal{J}|} I(\hat{y}_j \neq y_j)$$

2.2 Assessing Model Accuracy

Classification error

What is the BEST expected misclassification error?

$$\mathbb{E}|\mathcal{J}|^{-1} \sum_{j \in |\mathcal{J}|} I(\hat{y}_j \neq y_j)$$

Note that, from a statistics perspective, we view the world with randomness and probability. Therefore, given a determined x_i , we view the potential outcome y_i as random. Just like in regression, we assume y_i includes a random noise term. The same perspective applies to classification. That is, given x , we still assume y is random with s possible categories, say

$$\mathbb{P}(y = k|x) = \begin{cases} q_1, & k = 1 \\ q_2 & k = 2 \\ \vdots & \vdots \\ q_s & k = s \end{cases}$$

where $q_1 + \cdots + q_s = 1$.

2.2 Assessing Model Accuracy

OPTIMAL Classification: Bayes Classifier

Suppose that the distribution of labels are **KNOWN**

$$q_j(x) = \mathbb{P}(Y = j|X = x), \quad j \text{ is any label}$$

Bayes Classifier It is the **OPTIMAL** classifier: with minimal misclassification rate

$$f^{\text{Bayes}}(x) = \arg \max_{j \in \text{all labels}} q_j(x)$$

assigns the label with largest probability 

Example If only two labels: $\{-1, +1\}$

$$f^{\text{Bayes}}(x) = \begin{cases} +1, & \text{if } \mathbb{P}(Y = +1|X = x) > 0.5 \\ -1, & \text{otherwise} \end{cases}$$

2.2 Assessing Model Accuracy

OPTIMAL Classification: Bayes Classifier

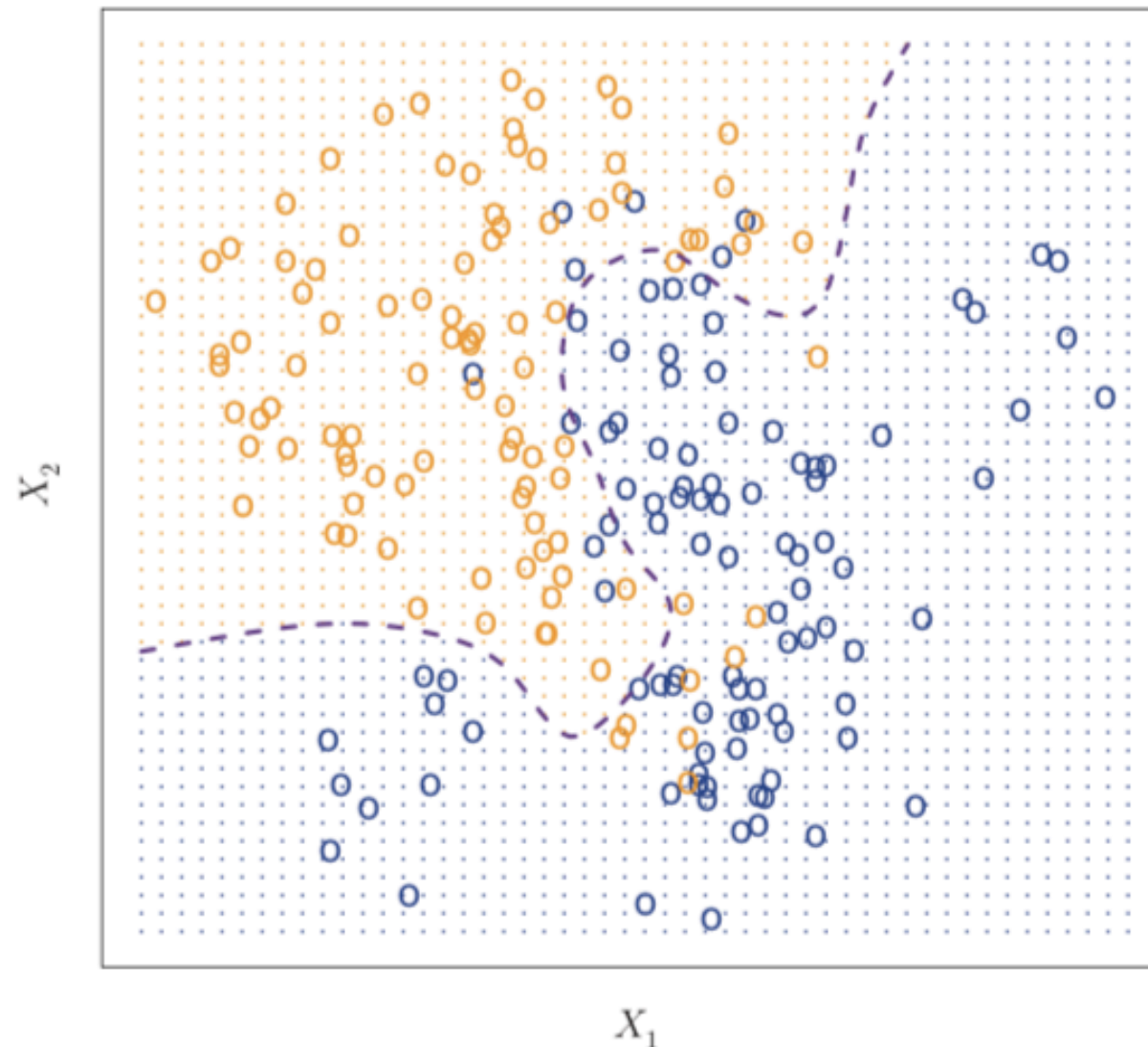


FIGURE 2.13. A simulated data set consisting of 100 observations in each of two groups, indicated in blue and in orange. The purple dashed line represents the Bayes decision boundary. The orange background grid indicates the region in which a test observation will be assigned to the orange class, and the blue background grid indicates the region in which a test observation will be assigned to the blue class.

2.2 Assessing Model Accuracy

KNN: K-nearest-neighbors

In practice, the distribution functions $q_j(x)$ of labels are impossible to KNOW.

KNN aims to estimate $q_j(x)$ by training data

$$\hat{q}_j(x_0) = K^{-1} \sum_{i \in \mathcal{N}_0} I(y_i = j)$$

K is a positive integer

$\mathcal{N}_0$ consists of top-K training data points closest to x_0

If data is more, then $\hat{q}_j(x_0)$ gets closer to $q_j(x_0)$

2.2 Assessing Model Accuracy

KNN: K-nearest-neighbors

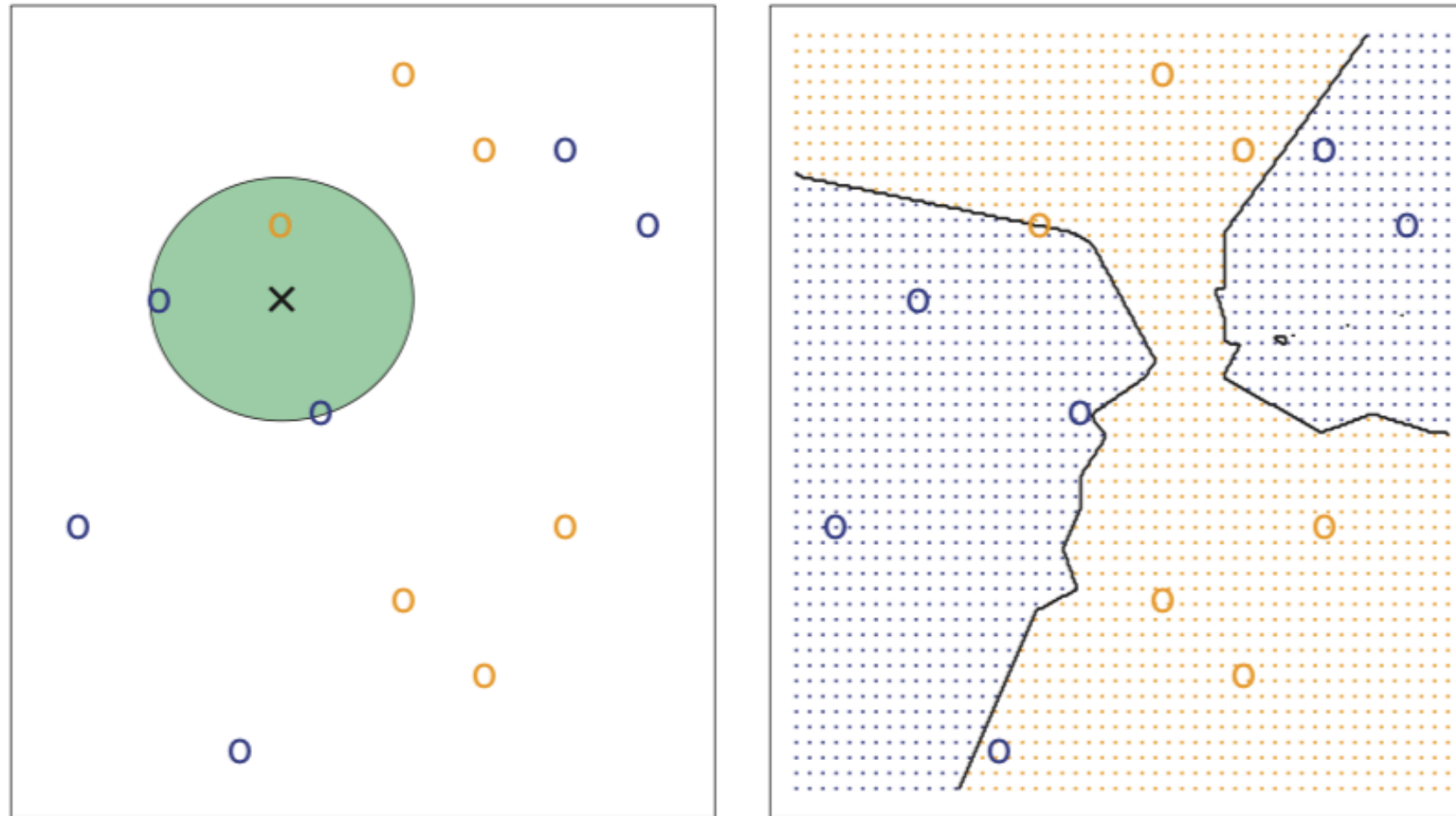


FIGURE 2.14. The KNN approach, using $K = 3$, is illustrated in a simple situation with six blue observations and six orange observations. Left: a test observation at which a predicted class label is desired is shown as a black cross. The three closest points to the test observation are identified, and it is predicted that the test observation belongs to the most commonly-occurring class, in this case blue. Right: The KNN decision boundary for this example is shown in black. The blue grid indicates the region in which a test observation will be assigned to the blue class, and the orange grid indicates the region in which it will be assigned to the orange class.

2.2 Assessing Model Accuracy

KNN: K-nearest-neighbors

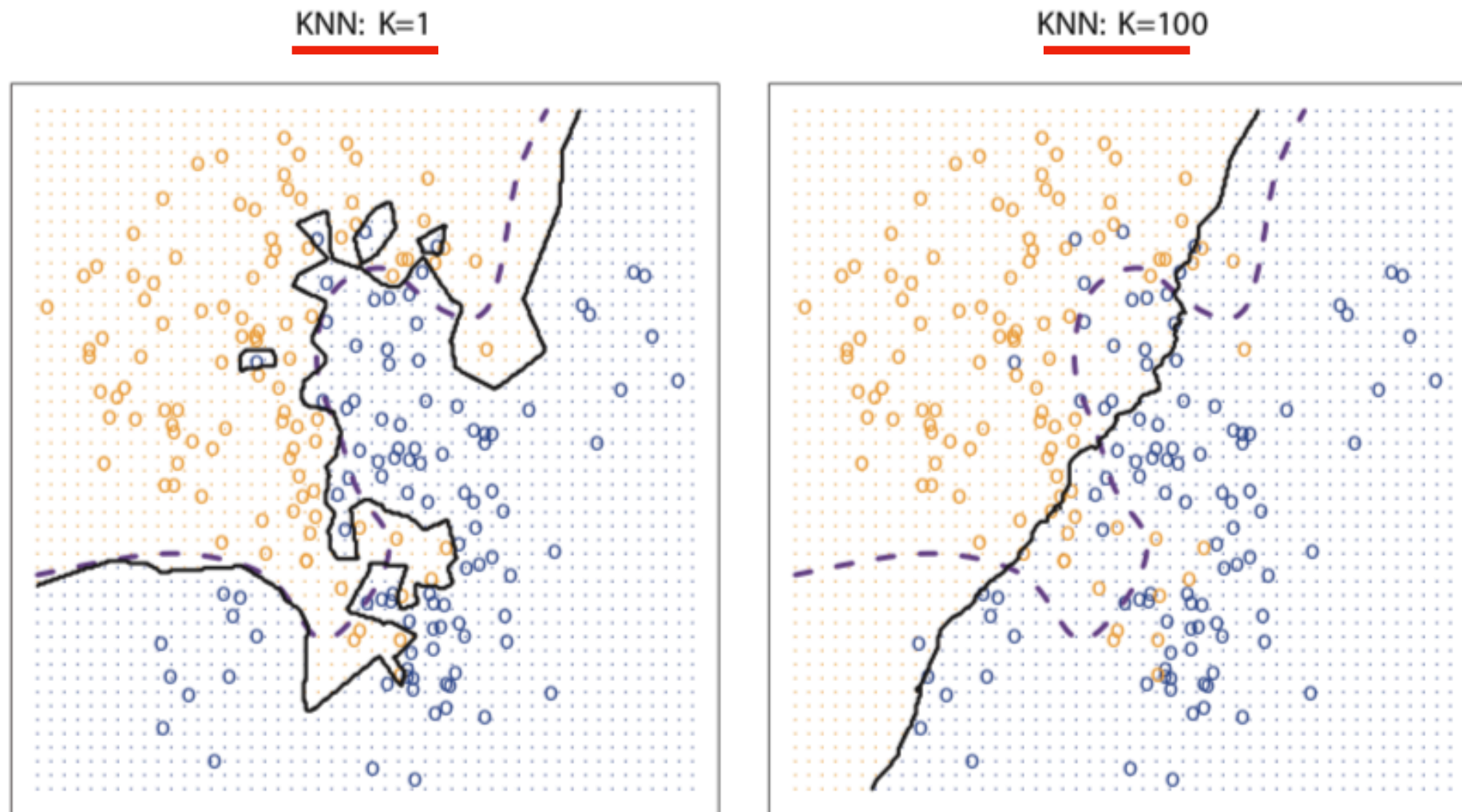


FIGURE 2.16. A comparison of the KNN decision boundaries (solid black curves) obtained using $K = 1$ and $K = 100$ on the data from Figure 2.13. With $K = 1$, the decision boundary is overly flexible, while with $K = 100$ it is not sufficiently flexible. The Bayes decision boundary is shown as a purple dashed line.

2.2 Assessing Model Accuracy

KNN: K-nearest-neighbors

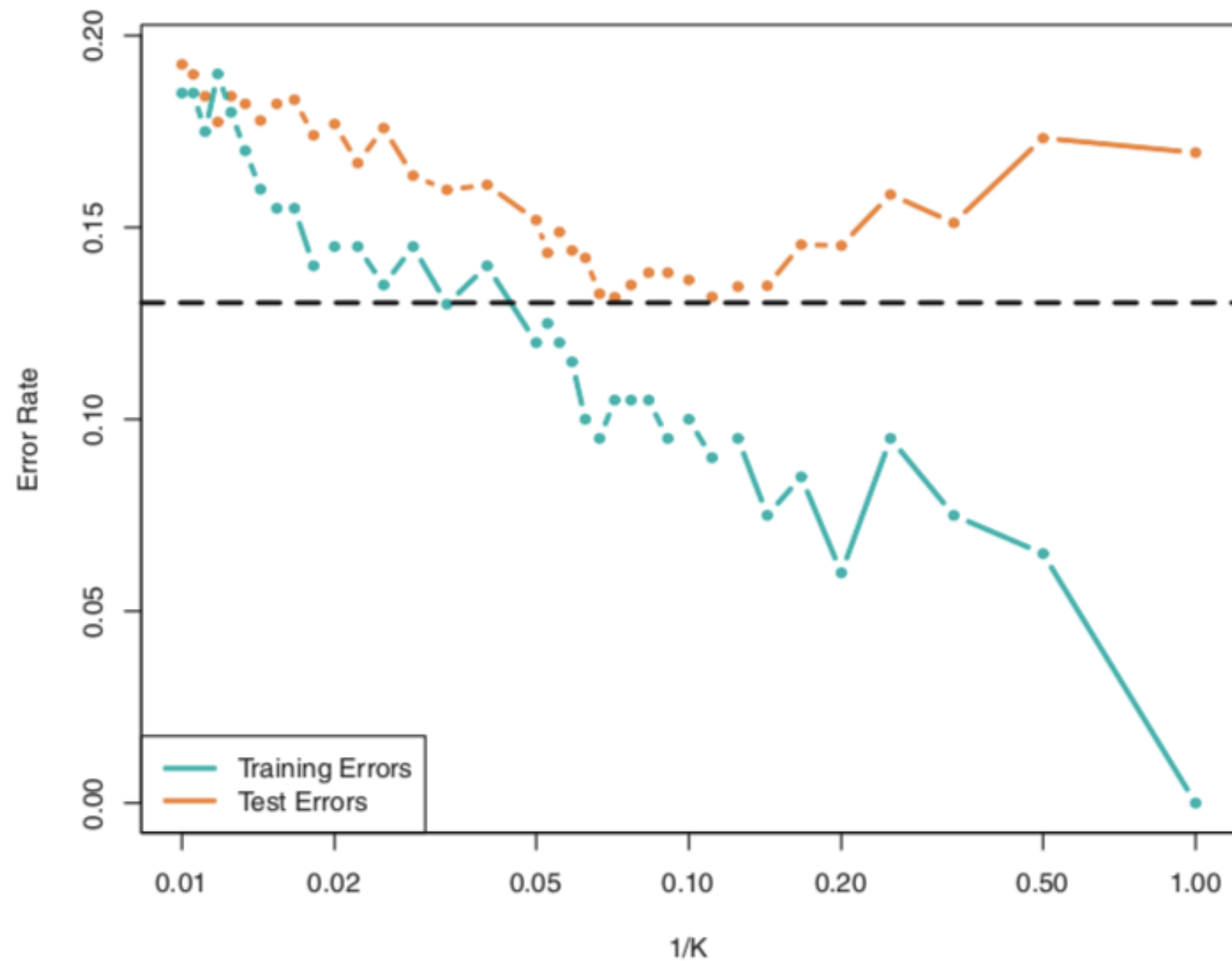


FIGURE 2.17. The KNN training error rate (blue, 200 observations) and test error rate (orange, 5,000 observations) on the data from Figure 2.13, as the level of flexibility (assessed using $1/K$) increases, or equivalently as the number of neighbors K decreases. The black dashed line indicates the Bayes error rate. The jumpiness of the curves is due to the small size of the training data set.

2.2 Assessing Model Accuracy

KNN: K-nearest-neighbors

```
##### KNN classification

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from matplotlib.colors import ListedColormap

# Generate synthetic data
np.random.seed(0)
X, y = make_classification(n_samples=300, n_features=2, n_redundant=0, n_informative=2, n_clusters_per_class=1)

# Plot the original data
plt.figure(figsize=(12, 4))
cmap_light = ListedColormap(['#FFAAAA', '#AAAAFF'])
cmap_bold = ListedColormap(['#FF0000', '#0000FF'])

# Define a function to plot decision boundaries
def plot_decision_boundary(clf, X, y, title, subplot):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                        np.arange(y_min, y_max, 0.1))
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.subplot(1, 4, subplot)
    plt.pcolormesh(xx, yy, Z, cmap=cmap_light, alpha=0.5)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold)
    plt.title(title)

# Plot original data
plt.subplot(1, 4, 1)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold)
plt.title('Original Data')

# K values to try
k_values = [1, 5, 10]
for i, k in enumerate(k_values, start=2):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X, y)
    plot_decision_boundary(knn, X, y, f'K={k}', subplot=i)

plt.tight_layout()
plt.show()

x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
```

